

This is a copy of a conversation between ChatGPT & Anonymous.



react interview topics list down

Here's a comprehensive list of **React interview topics**, organized by category and difficulty:

Fundamentals

- JSX and Virtual DOM
- Components (Function vs Class)
- Props vs State
- Component Lifecycle (Class & Hooks)
- Event Handling
- Conditional Rendering
- Lists and Keys
- Forms and Controlled Components

Hooks

- useState
- useEffect
- useContext
- useRef
- useMemo
- useCallback
- Custom Hooks
- Rules of Hooks

State Management

- Lifting State Up
- Context API
- Redux (Core Concepts, Actions, Reducers, Store)
- Redux Toolkit
- Zustand / Recoil / Jotai (if applicable)

Routing

- React Router (v6+)
 - Route, Link, Navigate, Outlet
 - Nested Routes
 - useParams, useNavigate
-

Advanced Concepts

- Higher-Order Components (HOC)
 - Render Props
 - Portals
 - Error Boundaries
 - Forward Refs
 - Code Splitting and Lazy Loading (**React.lazy**, **Suspense**)
 - Reconciliation and Fiber Architecture
 - Server Components (React 18+)
-

Testing

- Unit Testing (Jest)
 - Component Testing (React Testing Library)
 - Mocking APIs
 - Testing Hooks
-

Build Tools & Ecosystem

- Create React App vs Vite
 - Webpack / Babel Basics
 - ESLint & Prettier
 - Environment Variables
 - Linting and Formatting
-

APIs and Side Effects

- Fetching Data (fetch, axios)
 - useEffect for API Calls
 - Async/Await in React
 - Debouncing/Throttling
 - WebSockets / SSE (if needed)
-

UI/UX & Styling

- CSS Modules
- Styled-Components / Emotion
- Tailwind CSS
- Component Libraries (e.g., MUI, Ant Design)

Full-Stack Integration

- Connecting to Backends (Node.js, Firebase, etc.)
- JWT Auth Flow
- Protected Routes
- Form Validation (Formik, React Hook Form)

Performance Optimization

- React.memo
- useMemo and useCallback
- Lazy loading components
- Preventing re-renders

Behavioral / Conceptual

- Why React? Pros & Cons
- SPA vs MPA
- SSR vs CSR vs SSG
- When to use Redux vs Context API
- Common performance pitfalls

Let me know if you want these in a downloadable format (PDF or CSV) or sorted by beginner/intermediate/advanced!



explain with example:
useContext

Sure! Let's break down `useContext` in React with a **clear explanation** and a **simple working example**.

◆ What is useContext?

The `useContext` hook lets you **access values from a React Context** directly in a functional component—**without** needing to pass props down manually through every level of the component tree.

♦ Why use useContext?

To avoid "**prop drilling**"—passing data through many nested components that don't need it—by creating a shared **context** for global state or configuration.

✓ Basic Steps to Use useContext

1. **Create a Context** using `React.createContext()`
2. **Provide a value** using `<MyContext.Provider value={...}>`
3. **Consume the value** using `useContext(MyContext)`

Example: Theme Context

We'll build a simple example where a component accesses a shared **theme** using `useContext`.

1. Create ThemeContext.js

```
jsx

import React, { createContext } from 'react';

// Step 1: Create the context
const ThemeContext = createContext();

export default ThemeContext;
```

2. Create App.js (Provider and Consumer)

```
jsx

import React, { useState } from 'react';
import ThemeContext from './ThemeContext';
import Child from './Child';

function App() {
  const [theme, setTheme] = useState('light');

  return (
    // Step 2: Provide context value
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <div className={`app ${theme}`}>
        <h1>Current Theme: {theme}</h1>
        <Child />
      </div>
    </ThemeContext.Provider>
  );
}

export default App;
```

3. Create Child.js (Using useContext)

```
jsx

import React, { useContext } from 'react';
import ThemeContext from './ThemeContext';

function Child() {
  // Step 3: Use context
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <div>
      <p>Child Component - Theme: {theme}</p>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
        Toggle Theme
      </button>
    </div>
  );
}

export default Child;
```

Output Summary

- You see the **current theme** on screen.
- When you click "Toggle Theme", it switches between **light and dark**.
- The theme is shared between components using **useContext**, avoiding prop drilling.

Would you like a more advanced example with **useContext** + **useReducer** for state management?
Great! Let's break down these three **React Hooks**:

◆ **useRef, useMemo, and useCallback**

They all help with **performance optimization** or accessing **persistent values** in React.

1. **useRef**

◆ **What it does:**

- **Persists a mutable value** between renders.
- Doesn't cause re-renders when the value changes.
- Commonly used to access **DOM elements** directly.

✓ **Example: Focus an input field**

```
jsx

import React, { useRef } from 'react';

function UseRefExample() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus(); // Accessing DOM node
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Click button to focus" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

2. 🧠 useMemo

♦ What it does:

- **Memoizes** the result of an expensive calculation.
- Recalculates **only if dependencies change**.
- Avoids **unnecessary recalculations** on every render.

✅ Example: Expensive Calculation

```
jsx

import React, { useState, useMemo } from 'react';

function UseMemoExample() {
  const [number, setNumber] = useState(1);
  const [darkMode, setDarkMode] = useState(false);

  // Expensive function
  const slowFunction = (num) => {
    console.log('Computing...');
    for (let i = 0; i < 1e8; i++) {} // Simulate delay
    return num * 2;
  };

  const doubled = useMemo(() => slowFunction(number), [number]);

  return (
    <div style={{ background: darkMode ? '#333' : '#fff', color: darkMode ? '#fff' : '#000' }}>
      <input type="number" value={number} onChange={(e) => setNumber(parseInt(e.target.value))} />
      <button onClick={() => setDarkMode(!darkMode)}>Toggle Theme</button>
      <p>Doubled: {doubled}</p>
    </div>
  );
}
```

Without `useMemo`, toggling theme would re-run `slowFunction`.

3. 📄 useCallback

◆ What it does:

- Memoizes a **function** so it doesn't get recreated on every render.
- Useful when passing callbacks to child components (prevents unnecessary re-renders).

✓ Example: Memoized Function

```
jsx

import React, { useState, useCallback } from 'react';
import Child from './Child';

function UseCallbackExample() {
  const [count, setCount] = useState(0);
  const [toggle, setToggle] = useState(true);

  const increment = useCallback(() => {
    setCount((prev) => prev + 1);
  }, []);

  return (
    <div>
      <Child increment={increment} />
      <button onClick={() => setToggle(!toggle)}>Toggle</button>
      <p>Toggle state: {toggle.toString()}</p>
    </div>
  );
}

// Child.js
import React from 'react';

function Child({ increment }) {
  console.log('Child rendered');
  return <button onClick={increment}>Increment</button>;
}

export default React.memo(Child);
```

Without `useCallback`, the `increment` function would be recreated every render, causing the child to re-render unnecessarily.

🔄 Summary Table

Hook	Use Case	Causes Re-render?	Common Usage
<code>useRef</code>	DOM refs or persistent value	✗	Access input/element, store timers
<code>useMemo</code>	Memoize expensive values	✗	Slow functions, derived values
<code>useCallback</code>	Memoize functions	✗	Stable callbacks to children

Let me know if you want **real-world use cases** for these in a React project or if you'd like an interactive CodeSandbox link!



explain redux with example

Sure! Let's break down **Redux** in a simple, practical way—with explanation + **working example** using React.

◆ What is Redux?

Redux is a predictable **state container** for JavaScript apps. It's often used with **React** to manage global state (e.g., user data, theme, cart, etc.).

◆ Why Redux?

- Avoids **prop drilling** (passing props through many levels).
- Centralized state = easier debugging and maintenance.
- Middleware support (e.g., Redux Thunk) for async logic.

🔧 Key Redux Concepts

Concept	Purpose
Store	Holds the state
Action	Describes what happened
Reducer	Changes state based on action
Dispatch	Sends action to reducer
useSelector	Reads state in a component
useDispatch	Triggers actions from components

✅ Redux Example (Counter App)

We'll build a simple app that uses Redux to increment/decrement a counter.

1 Install Dependencies

```
bash

npm install @reduxjs/toolkit react-redux
```

2 Create Redux Slice (counterSlice.js)

```
js

// src/redux/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
```



```

initialState: { value: 0 },
reducers: {
  increment: state => { state.value += 1 },
  decrement: state => { state.value -= 1 },
  reset: state => { state.value = 0 },
},
});

export const { increment, decrement, reset } = counterSlice.actions;
export default counterSlice.reducer;

```

3 Create Store (store.js)

```

js

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});

export default store;

```

4 Provide the Store (index.js or main.jsx)

```

jsx

// src/index.js
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import { Provider } from 'react-redux';
import store from './redux/store';

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);

```

5 Use Redux in a Component (Counter.js)

```

jsx

import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, reset } from './redux/counterSlice';

function Counter() {
  const count = useSelector(state => state.counter.value); // read state
  const dispatch = useDispatch(); // to send actions

  return (
    <div>
      <h2>Counter: {count}</h2>
      <button onClick={() => dispatch(increment())}> + </button>
    </div>
  );
}

```

```

    <button onClick={() => dispatch(decrement())}> - </button>
    <button onClick={() => dispatch(reset())}> Reset </button>
  </div>
);
}

export default Counter;

```

6 Main App

```

jsx

// src/App.js
import React from 'react';
import Counter from './Counter';

function App() {
  return (
    <div>
      <h1>Redux Counter App</h1>
      <Counter />
    </div>
  );
}

export default App;

```

Output Behavior

- Click +, it increments.
- Click -, it decrements.
- Click **Reset**, it resets to 0.
- All state is managed **globally** via Redux.

Summary

- Redux gives you **global, centralized state**.
- Redux Toolkit makes setup much simpler.
- Use **useSelector** to read, **useDispatch** to update.

Would you like this same app extended with **async Redux Thunk** (e.g., fetch API data)?



explain with example:
Higher-Order Components (HOC)

Sure! Let's dive into **Higher-Order Components (HOC)** in React with a clear explanation and example.

◆ What is a Higher-Order Component (HOC)?

A **Higher-Order Component** is a function that takes a component and returns a new component.

📌 Definition:

A HOC is a pattern used in React to **reuse component logic**.

It's similar to a higher-order function in JavaScript.

🔧 Syntax

```
jsx

const EnhancedComponent = higherOrderComponent(WrappedComponent);
```

✓ Use Cases

- Code reuse (e.g., auth checks, loading states)
- Adding props or logic to components
- Handling cross-cutting concerns

🔪 Example: Add Loading Spinner with HOC

We'll create a HOC that shows a loading message while the data is being fetched.

1 Basic Component

```
jsx

function UserList({ users }) {
  return (
    <ul>
      {users.map((user, i) => <li key={i}>{user}</li>)}
    </ul>
  );
}
```

2 Higher-Order Component: withLoader

```
jsx

function withLoader(WrappedComponent) {
  return function EnhancedComponent({ isLoading, ...props }) {
    if (isLoading) return <p>Loading...</p>;
    return <WrappedComponent {...props} />;
  };
}
```

3 Wrap Component Using HOC

```
jsx

const UserListWithLoader = withLoader(UserList);
```

4 Use It in App

```
jsx

function App() {
  const [loading, setLoading] = React.useState(true);
  const [users, setUsers] = React.useState([]);

  React.useEffect(() => {
    setTimeout(() => {
      setUsers(['Alice', 'Bob', 'Charlie']);
      setLoading(false);
    }, 2000); // Simulated fetch delay
  }, []);

  return (
    <div>
      <h1>Users</h1>
      <UserListWithLoader isLoading={loading} users={users} />
    </div>
  );
}
```

Summary

Concept	Description
HOC Purpose	Reuse component logic
Accepts	A component
Returns	A new enhanced component
Common Uses	Auth guards, loading indicators, themes

Note

- **Don't use HOC inside render** – it breaks React rules.
- HOCs **don't modify the original component**, just wrap it.

Would you like an example of HOC for **authentication guard** (`withAuth`) or for **logging props**?



explain with example:
useMemo and useCallback

Absolutely! Let's break down `useMemo` and `useCallback` in React with clear explanations and working examples.

`useMemo` VS `useCallback`

Hook	Purpose	Returns	Use Case
<code>useMemo</code>	Memoizes a value	Memoized value	Expensive calculations
<code>useCallback</code>	Memoizes a function	Memoized func	Stable function references

1. `useMemo` Example – Avoid Expensive Recalculation

What it does:

Caches the result of a **slow or expensive function** and only recalculates when dependencies change.

Example:

```
jsx

import React, { useState, useMemo } from 'react';

function UseMemoExample() {
  const [num, setNum] = useState(1);
  const [dark, setDark] = useState(false);

  // Expensive calculation
  const slowSquare = (n) => {
    console.log('Running slow function...');
    for (let i = 0; i < 1e8; i++) {} // simulate delay
    return n * n;
  };

  const squared = useMemo(() => slowSquare(num), [num]); // only recalculates if `num` changes

  const themeStyles = {
    backgroundColor: dark ? '#333' : '#fff',
    color: dark ? '#fff' : '#000',
    padding: '1rem',
    marginTop: '1rem'
  };

  return (
    <div>
      <input
        type="number"
        value={num}
        onChange={(e) => setNum(parseInt(e.target.value) || 0)}
      />
      <button onClick={() => setDark(prev => !prev)}>
        Toggle Theme
      </button>
      <div style={themeStyles}>
        Squared value: {squared}
      </div>
    </div>
  );
}
```

Why it matters:

Without `useMemo`, the slow function runs **on every render**, even when only the theme changes.

✓ 2. useCallback Example – Avoid Unnecessary Re-Renders

🧠 What it does:

Memoizes a **function**, so it's not recreated on every render unless dependencies change.

🔧 Example with Child Component:

```
jsx

import React, { useState, useCallback } from 'react';
import Child from './Child';

function UseCallbackExample() {
  const [count, setCount] = useState(0);
  const [dark, setDark] = useState(false);

  const increment = useCallback(() => {
    setCount(prev => prev + 1);
  }, []); // function only created once

  const theme = {
    backgroundColor: dark ? '#333' : '#fff',
    color: dark ? '#fff' : '#000',
    padding: '1rem'
  };

  return (
    <div style={theme}>
      <h2>Count: {count}</h2>
      <Child onClick={increment} />
      <button onClick={() => setDark(!dark)}>Toggle Theme</button>
    </div>
  );
}
```

Child.js

```
jsx

import React from 'react';

function Child({ onClick }) {
  console.log('Child rendered');
  return <button onClick={onClick}>Increment from Child</button>;
}

export default React.memo(Child); // Prevents re-render unless props change
```

✓ Why it matters:

Without `useCallback`, the `onClick` function is **new on every render**, so `Child` re-renders unnecessarily. With `React.memo` + `useCallback`, you **prevent that extra render**.

🎯 Summary

Hook	When to Use
<code>useMemo</code>	For expensive computations
<code>useCallback</code>	For passing stable functions to children

Let me know if you want a **real-world use case**, like a memoized API filter function or optimized list rendering!